kurv

ENTERPRISE DATA PLATFORM · ENGINEERING REFERENCE

KurvPay *EDP* The Essentials

Condensed · v2

The context-dense edition — every chapter of the full reference, distilled to its load-bearing facts, decisions, and one diagram apiece. Business, engineering, and the agentic factory that builds it.

32	63	8	60
TSYS TYPES	AI SUBAGENTS	FACTORY STAGES	LESSONS

CLIENT
Kurv (KurvCollections)

DELIVERY
Pythian

STACK
Python 3.12 · AWS Lambda · Databricks Lakeflow

COMPILED
2026-07-24

Condensed companion to KurvPay EDP — The Complete System. Metrics measured at compile time.

CONDENSED REFERENCE · V2

One repository, two products

KurvPay EDP is two products in one repo: a **payments data platform** (32 TSYS types → governed Delta Lake via Python / Lambda / Lakeflow) and the **agentic factory** that builds it (62 instantiated subagents, 29 commands, 12 skills, 10 rules, a 60-lesson flywheel). This is the condensed reference — the state of the platform on 2026-07-24.

32	42×	~98%	~8h
TSYS TYPES · ALL ONBOARDED	FASTER THAN T-SQL	COST CUT	PER FILE (AGENTIC)

THE BUSINESS

Pythian modernizes payment-data processing for Kurv. Legacy T-SQL is retired — and is the golden ground truth new pipelines now validate against field-for-field.

THE ENGINEERING

Four-layer parsers (model / parser / schema / writer) in isolated Lambdas → Lakeflow Bronze. Decimal-only money, COBOL overpunch, UUID hierarchy.

THE AGENTIC SYSTEM

62 subagents (28 parser experts), grounded by a machine-readable KB + live MCP docs before touching a financial parser.

THE DARK FACTORY

An 8-stage gated pipeline + 60-lesson flywheel, fired unattended over a queue by loop-engine — a human only at the deploy gate. 26 of 28 matched types now golden-signed vs legacy.

Contents

01 The Business

Payments, TSYS, and the modernization mandate

Pythian is modernizing payment-data processing for **Kurv (KurvCollections)**. Their processor **TSYS** ships merchant data as SFTP batch files; a legacy **T-SQL/SSIS** stack parses them on SQL Server. KurvPay EDP replaces that with **Python + AWS Lambda + Databricks Lakeflow** — Bronze-only today, Silver/Gold on return (Silver load-logic now being spec'd ahead of that return).

ESSENTIALS

- Client **Kurv**; delivery by **Pythian**. Kicked off **2025-11-25** (16-wk, later ~20-wk SOW).
- Central risk is **financial correctness under translation** — a wrong field offset or sign makes plausible-but-wrong money that structural tests miss.
- Legacy is the **golden ground truth**: `python -m src.cli match` now diffs new output against the MSSQL/SSIS dumps field-for-field before any sign-off — **26 of 28** matched types signed.
- Grew from **8 core types** to the full **32-type surface** (`configs/func-types.sh`), all onboarded to Bronze.
- Key people: **Nicole Kennedy** (Kurv interim CTO), **Keith Evans** (Kurv architect, owns Silver logic), **Jeff DeVerter** (Pythian Field CTO), **Kenny Shaw**, **Luan Moreno** (Pythian lead).

Why migrate — the business case

The legacy stack is a single point of failure (NFR-001) and a velocity bottleneck. Measured in `docs/perf-tsys-tsql-aws-lambda.md` :

Metric	Legacy T-SQL/SSIS	AWS Lambda	Gain
One file	~42 s	~1 s	**42×**
1,000 files	~11.6 h (serial)	~2 s (parallel)	**massive**
Parser LOC (4 types)	~7,600	~1,555	**6.8× less**
Deploy	~4 h, 6+ people	~15 min, 1 dev	**16×**
Recovery	~2 h	~16 min (DLQ)	**8×**
Monthly cost	~$2,500	~$25	**~98%**

The TSYS file surface (32 types)

Each file carries a slice of the merchant-processing lifecycle:

- **Authorizations / transactions** — ADF (auth detail), TDDF (daily detail, dual sign), D256 (256-byte settlement, COBOL overpunch)
- **Merchant reference / statements** — MDI (117 tab fields), Merchant Statements (XML blob), addr, agt, individual-plan
- **Profitability / residuals** — PPM family (ppm-key-structure / ppm-key-value / merchant-dda / erg-extract / pricing)
- **Money movement** — ACH (+ holdover), debit-recon, rf-merchant / rf-activity, charge-journal, draftc
- **Disputes / fraud / chargebacks** — TC40 (Visa fraud), TC33, TQR4 (MC dispute), CBRT, downgrade (3-way network)
- **Reconciliation / settlement** — gl-recon / gl-interface, visa-eod, **discover-ari** (hardest: 17 tables, 40 overpunch money fields), aob-grrcnd-det, mra

The SOW arc & the agentic pitch

The scope expanded under time pressure: the close-out sprint added **PPM, Merchant Statements, ACH** (on-site handoff 2026-03-24), then Silver/Gold for existing files was pulled into the current engagement on the **2026-04-15** discovery call. Strategy: **V1** quick-win Silver/Gold (Daily MDI, Merchant Statements, TDDF) + **V2** processor-agnostic architecture; Keith designs in SQL Server, Luan translates to Lakeflow and runs `KT-01..05` sessions. The agentic story is itself the product: manual translation runs **2–3 months/file**; the Claude Code spec-driven method runs **~8 h/file** — the pitch Pythian is selling upward.

```
SFTP → S3 LANDING → AWS LAMBDA → S3 STAGE → DATABRICKS LAKEFLOW
(TSYS)  (raw files)  (parse→Parquet)  (Parquet)  (Bronze · Unity Catalog)
                                                  └ Silver/Gold on return (spec'd)
```

02 History & Governance

The arc, the tests, the decision records

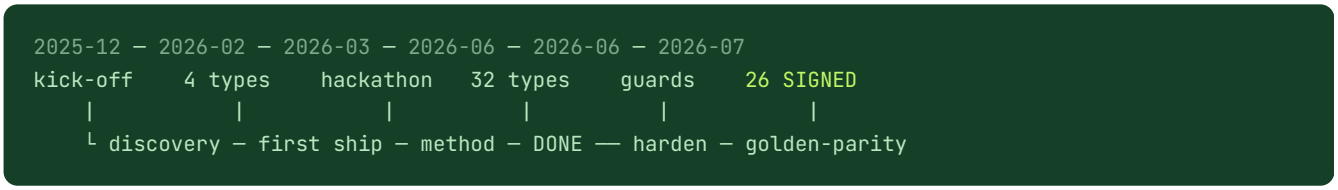
From 4 types (2026-02-02) to the full 32-type surface (2026-06-29), then a field-for-field **golden-parity crank** (July 2026) — the platform grew under strict test, ADR, and doc-freshness discipline.

ESSENTIALS

- **666 commits**, ~210 branches; **32 of 32** types `status: complete`; **26 of 28** golden-matched types `signed` (2 stalled on fixture defects).
- **~2,800 tests**, **80% coverage gate**, ruff lint + bandit SAST, **8-gate preflight** before any PR (Gate 8 = match-config lint, added 2026-07-15).
- **PII-masking ADR (2026-04-18)**: PII stored **raw in Bronze**; RBAC at Silver/Gold; `mask_pan()` deprecated.
- **DOCS-WITH-CHANGE + doc-freshness**: a doc-visible change ships its README edit in the same diff; counts are measured, never guessed.
- **Living vs historical** doctrine: notes / ADRs / decks are timestamped records — edited only via supersession.

Project timeline

- **2025-11-25** — external kick-off (replace SQL Server)
- **2026-02-02** — first 4 types shipped (adf, mdi, tddf, d256)
- **2026-03-11** — hackathon: the agentic methodology framed as a strategic asset
- **2026-03-30 → 04-21** — Lambda migration; SOW close-out (Merchant Statements, ACH, PPM)
- **2026-04-18** — PII-masking ADR ratified
- **2026-06-08** — Bronze-only reset (Silver/Gold demos removed, kept in history)
- **2026-06-29** — full 32-type Dark Factory surface complete; process guards added
- **2026-07-06** — `/tsys:auto` renamed `/tsys:onboard` (uniform `--auto` mode)
- **2026-07-13 → 07-16** — golden-parity sign-off crank: **26 signed / 2 stalled**; d256 16/16; multi-stem match
- **2026-07-22** — Silver-layer load procs + DDL spec'd for 11 types (ahead of Silver's return)



Testing & ADRs

Pytest with per-type markers (`-m tddf`, `-m unit`); coverage gate at 80%; the `preflight-validate.sh` **8 gates** (tests, ruff lint, bandit SAST, build-copy-src + import, handler parity, source parity, e2e smoke, and match-config lint) must pass before a PR. **ADRs** in `docs/adrs/` govern cross-cutting decisions — the load-bearing one is PII-masking: Bronze faithfully stores PAN/SSN/bank-account bytes raw (a Bronze layer is a raw projection of the file), and column-level security is enforced as Unity Catalog RBAC dynamic views at Silver/Gold, not in the parser.

03 The Parsing Engine

T-SQL → Python: models, parsers, schemas, writers

Every type is a four-layer set — **frozen-dataclass model → generator parser → PyArrow schema → Parquet writer** — auto-registered at import and packaged into an isolated Lambda. Financial correctness is enforced *structurally*.

ESSENTIALS

- **32 parsers auto-registered** at import via `PARSER_REGISTRY` (34 registry keys incl. variant dispatch); 35 models; 32 schemas + 33 writers + 32 handlers.
- Money is always `Decimal`, **never float**; `parse_money(raw, implied_decimal=?)` — wrong flag = the **100×** scaling bug (L-44).
- **COBOL overpunch** (`utils/cobol.py`): sign lives in the last byte (`}` = −0, `{` = +0, A-I = +1..9, J-R = −1..9).
- `substring_slice` applies the 1-indexed-SQL → 0-indexed-Python `-1` once (L-15 / G-04).
- `ParsingContext` mints UUIDs per hierarchy level — the `SCOPE_IDENTITY()` replacement; **zero-row Parquet placeholders** (L-27) keep empty tables from breaking DLT.
- **STRICT no-comments** (Google docstrings only); PII raw in Bronze; `defusedxml` for Merchant Statements.

The four layers

Layer	Path	Job
Model	`models/{type}.py`	frozen `@dataclass`; `from_fixed_width()` field extraction
Parser	`parsers/{type}_parser.py`	`@register_parser`; `parse()` generator → `Iterator[Record]`
Schema	`schemas/{type}_schema.py`	PyArrow schema per output table + SQL name map
Writer	`writers/{type}_writer.py`	classify by `isinstance`, write per-table `.parquet`
Handler	`handlers/{type}_handler.py`	Lambda entry: S3 get → parse → write → S3 put

Parsers **stream** line-by-line (`yield`) so peak RAM stays flat; they catch only `(ValueError, KeyError, IndexError)` per line, skip-and-continue, and track quality via `ParsingStats`.

```
raw line → detect record type → model.from_fixed_width(data, ctx) → yield record
             ParsingContext — mints file_uuid / parent UUIDs / record_seq
                                    ▼
        writer: isinstance → per-table Parquet (+ zero-row placeholder if empty)
```

FlowCheck CLI & the src/ → functions/ boundary

`src/core/` is the **canonical source** (develop + test here); `scripts/build-copy-src.sh` promotes it into `functions/*/` (rewriting `src.core.` → `src.`, AST-generating `__init__.py`) — `functions/` is **never edited directly**. FlowCheck (`src/cli/`, typer + rich) debugs the pipeline: `trace`, `status`, `inspect`, `logs`, `profile`, `compare`, `match` (golden-parity), `observe` (token/cost telemetry), `xref` (load lookup tables).

04 Cloud Infrastructure

AWS Lambda · Databricks Lakeflow · CI/CD

S3-triggered Lambdas parse to Parquet; one Lakeflow DLT pipeline per type ingests it into a Unity Catalog Medallion (Bronze-only today). Azure Pipelines gate every PR and deploy.

ESSENTIALS

- **Per-type isolated Lambdas** — 32 deployed (SAM templates, least-privilege IAM, adaptive S3 retry ×3, sanitized errors, DLQ).
- Data flow: SFTP → S3 Landing → Lambda → S3 Stage (Parquet) → Lakeflow Bronze.
- **One DLT pipeline per type** via Databricks Asset Bundles (DABs); Unity Catalog governance; dev/stg/prd.
- CI (`ci/`): **8-gate preflight** on PR. CD (`cd/`): Lambda deploy on merge to dev/master.
- Config (`configs/`): `.envrc` + `aws-env.sh` + `databricks-env.sh`, auto-loaded by direnv; **source-before-run**; no hardcoded IDs.

The deploy pipeline

The promotion path is a single gate: `src/` is tested once, then `build-copy-src.sh` fans it out into per-type Lambda packages, which SAM deploys. Databricks pipelines deploy from the shared bundle (`pipelines/databricks.yml`) — deployed in desired-state so one onboarding's deploy never deletes another's pipeline (L-38: never `--auto-approve`).

```
src/core/ —build-copy-src.sh—→ functions/{type}/ —SAM—→ AWS Lambda (per type)
  (1 test run)    (rewrite imports,          (deploy package)        |   S3 trigger
                   AST __init__.py)                                  ▼
pipelines/ —DABs—→ Databricks Lakeflow DLT —→ Bronze Delta (Unity Catalog)
```

05 The Agent Fleet

62 specialized subagents + the Knowledge Base

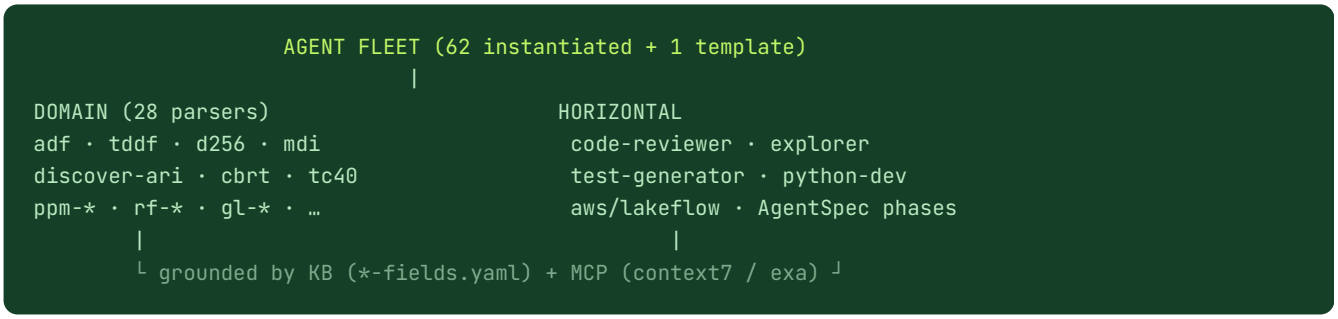
> An agent here is a specialized subagent with its own system prompt, tools, and worked examples. 28 of the fleet are **per-file-type parser experts** that encode money / dispatch / PII rules as executable knowledge.

ESSENTIALS

- **63 agent definition files** — 62 instantiated agents + 1 `tsys/` new-type scaffold template — across ai-ml, aws, code-quality, communication, data-engineering, dev, exploration, tsys, workflow.
- **28 per-type parser specialists** (adf-parser, tddf-parser, d256-parser, discover-ari-parser, …) each carrying its type's lessons.
- **Agreement Matrix**: agents validate against the KB + live docs (MCP: context7, exa) before writing a financial parser.
- Cross-cutting: code-reviewer, codebase-explorer, test-generator, python-developer, lakeflow/aws specialists, meeting agents, AgentSpec phase agents.
- KB (`.claude/kb/`): machine-readable `{type}-fields.yaml` specs ground every parser agent in field positions & revisions.

Fleet taxonomy

The fleet splits into **domain specialists** (the 28 parser experts — deep, narrow, one file type each) and **horizontal specialists** (review, exploration, testing, cloud, orchestration). The AgentSpec **phase agents** (brainstorm → define → design → build → ship → iterate) drive generic spec-driven development; parser agents plug into the TSYS factory.



06 Commands, Skills & Rules

The composable agentic tooling layer

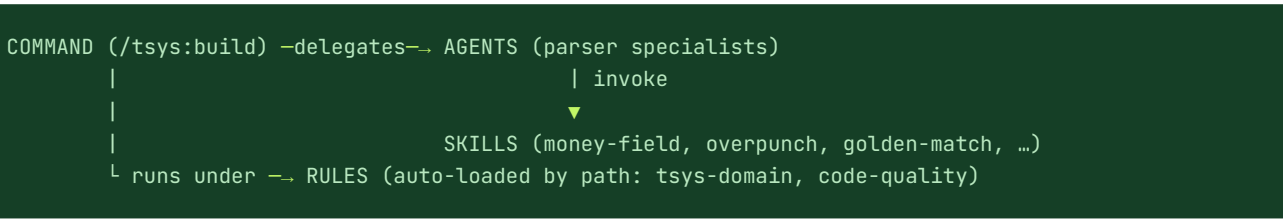
Four composable layers: **29 commands** orchestrate, **12 skills** are callable capabilities, **10 path-scoped rules** load domain law by file path, and agents do the work.

ESSENTIALS

- **tsys:* commands (12)** — the factory: `onboard` (full pipeline; formerly `auto`), `build` (8-stage), `define / design / ship`, `change` (spec propagation), `match` (golden), `validate`, `xref`, `status`, `setup`.
- **workflow:* (7)** — generic AgentSpec SDD (brainstorm / define / design / build / iterate / ship / create-pr). Plus communication (4), core (4), knowledge (1), review (1).
- **12 skills** — the 5 TSYS decision-tree skills `tsys-money-field`, `tsys-cobol-overpunch`, `tsys-positional-hierarchical`, `tsys-reconciliation`, `tsys-golden-match` + `loop-engine`, `task-spec`, `visual-explainer`, `pdf`, `kurv-style-guide`, `html-to-pdf`, `excalidraw-diagram`.
- **10 rules** auto-load by path glob — code-quality, architecture, tsys-domain, lakeflow, testing, configs, tsys-framework, agents-and-commands, documentation, doc-freshness.

How the layers compose

A skill is progressive-disclosure: a `SKILL.md` loads only when invoked. A rule is ambient — editing `src/core/parsers/**` auto-loads `tsys-domain.md`. The five TSYS skills encode the hardest financial decisions (the money 100× tree, the overpunch decoder, the positional-hierarchical builder, the reconciliation-basis picker, the golden-diff triage) so they are structurally hard to get wrong.



07 The TSYS & Dark Factory

The self-improving onboarding assembly line

An **8-stage gated pipeline** drives each type from spec to deployed Bronze. A **60-lesson memory flywheel** sharpens every run; recurring lessons become skills. The **Dark Factory** fires the whole pipeline unattended over a queue — a human touches it only at the deploy gate — and a field-for-field **golden-parity crank** proves the output matches legacy.

ESSENTIALS

- **8 stages**: scaffold → kb-agent → models → tests → preflight → lambda → lakeflow → e2e, gated between each; `09-deploy` is a manual runbook, not a stage.
- `status.yaml` is the build state machine (pending / in_progress / complete); `match-state.yaml` is the parity ledger (signed / stalled); `memory.md` holds **60 L-NN + 9 G-NN** lessons.
- `/tsys:onboard` sequences the pipeline unattended with one **L-23 deploy gate** (waivable via `--unattended`); `/tsys:change` propagates spec revisions diff-first behind a destructive-change gate.
- **Dark Factory** = the `loop-engine` skill (queue + self-firing clock + per-tick action + stall-and-continue) firing `/tsys:onboard {type} --unattended` over the 32-type surface.
- **Golden-parity** (`/tsys:match`): a 4-stage diff vs MSSQL/SSIS — **26 of 28 signed**, ~160k rows value-for-value, 9 cited parser fixes, 0 uncited edits.
- **Process guards**: the freshest lessons become deterministic gates (S3-prefix L-56, pricing-drift, all-zero-Bronze).

Why a factory at all

Onboarding one TSYS type by hand is a 2–3 month T-SQL archaeology project: read a stored proc, decode fixed-width offsets and COBOL sign conventions, rebuild the schema, wire a Lambda, stand up a pipeline, and prove the numbers. The Dark Factory turns that into a **repeatable, gated, unattended assembly line**: the same eight stages run for every type, each gate refuses to advance on failure, the memory ledger carries every hard-won lesson forward, and the only human decision is a single "yes" at the cloud-deploy boundary. The 8-to-32-type expansion (finished 2026-06-29) was run this way in batches; the July sign-off crank then proved, type by type, that the output is byte-faithful to the system it replaces.

The 8-stage gated pipeline

Each stage is a discrete prompt (`.claude/tsys/prompts/01-08`) with an explicit **gate** — a verification that must pass before the next stage runs. Stages 02–08 each open with a Lessons-Context block citing the relevant `L-NN` entries, so the memory flywheel feeds directly into the build.

#	Stage	What it produces · the gate
1	`scaffold`	Type skeleton (model/parser/schema/writer/handler stubs), func-types + registry wiring. Gate: files exist, imports resolve.
2	`kb-agent`	The KB spec (`{type}-fields.yaml`) + the per-type parser agent, grounded in the proc/DDL. Gate: field positions cited to spec.
3	`models`	Frozen dataclasses + parser + schema + writer. Gate: parses the sample, Stage-3 baselines captured before tests (L-07).
4	`tests`	Unit + integration tests (model, writer, routing). Gate: green + coverage ≥ 80%.
5	`preflight`	The full 8-gate `preflight-validate.sh`. Gate: all gates pass (lint, SAST, build, parity, e2e, match-config).
6	`lambda`	SAM template + least-privilege IAM + samconfig. Gate: `sam build` + local invoke on the sample.
7	`lakeflow`	Bronze DLT notebook + DABs resource. Gate: `bundle validate`; read_files glob matches the writer's actual output (L-36).
8	`e2e`	Local end-to-end reparse + FlowCheck trace + row reconciliation. Gate: detail count == the reconciliation basis.

```
scaffold →kb-agent→models→tests→preflight→lambda→lakeflow→e2e  ──→ [09-deploy]
 [gate]   [gate]   [gate]  [gate]   [gate]    [DEPLOY]  [gate]   [ship]        manual runbook
   └── every gate must pass before the next stage runs; a fail stalls the type ──┘
```

Stage 8 is where the `/tsys:build` factory finishes. Cloud deployment (`sam deploy` + `databricks bundle deploy`) is a separate manual runbook (`09-deploy.md`) — never a build stage — which encodes L-23: **cloud E2E is human-invoked**. After a green build, `/tsys:ship` writes the archive artifacts and appends new lessons to `memory.md`.

The gates — where correctness is enforced

The factory is only as safe as its gates. There are four kinds, and only one of them is a human gate.

1 · STAGE GATES (BETWEEN EVERY STAGE)

Each of the eight stages ends in a verification that must pass before the next begins. A failed stage **stalls that type** with the exact stage + reason recorded — it never silently advances with a half-built layer. This is what makes an unattended run safe: a broken parser can't reach a Lambda deploy.

2 · THE 8-GATE PREFLIGHT (STAGE 5, AND AGAIN IN CI)

`scripts/preflight-validate.sh` runs eight independent gates: (1) unit + integration tests, (2) ruff lint, (3) bandit SAST + secrets grep, (4) build-copy-src + import validation, (5) handler parity, (6) source-file parity, (7) e2e smoke (skippable in CI with `--skip-e2e`), and (8) **match-config lint** — the newest, which audits every golden-parity config for placeholder join keys and `amount_columns ∩ join_keys` overlap.

3 · THE L-23 DEPLOY GATE (THE ONE HUMAN TOUCH)

Cloud infrastructure requires exactly one human "yes" per run. `/tsys:onboard` pauses at the deploy gate and never auto-fires `sam deploy` / `databricks bundle deploy` without it. The **sanctioned waiver** is `--unattended` (added 2026-06-10, once the flow was proven across ~9 green runs): the conscious opt-in is the approval, given once at invocation instead of once at the gate — enabling overnight batch onboarding.

4 · CORRECTNESS GATES THAT HOLD EVEN UNDER `--UNATTENDED`

Waiving the human gate does not weaken the correctness gates. Under `--unattended` these still fire and stall the type: the Step-0 collision STOP (won't re-onboard a complete type), every stage-gate failure, the Phase-7.5 validate, the **destructive-bundle-delete refusal** (a proposed pipeline delete stalls the type — never `--auto-approve`, L-38), and auth-failure-as-deploy-failure. `/tsys:change` adds a fifth: a **destructive-change gate** — table renames / drops and column drops need one explicit approval; reversible field edits auto-propagate.

```
EVERY RUN:  stage gates (×8)  +  preflight (8 gates)  +  correctness gates
ONE HUMAN:  L-23 deploy gate  ──  waivable ONLY via explicit --unattended
NEVER WAIVED: collision STOP · stage fails · validate · bundle-delete refusal · auth-fail
```

The morning loop — unattended, self-firing

An LLM agent can't loop on its own; it only acts when invoked. The `loop-engine` skill supplies the missing pieces so a batch runs while the operator sleeps: **a queue (the memory) + a self-firing clock (the re-invoker) + a per-tick action (the work) + graceful failure (stall-and-continue)**. The Dark Factory is its first consumer.

Part	What it is	Dark Factory instance
Goal-state file	A queue: one row per item with a `status` column	`tasks/dark-factory-queue.md`
Per-tick action	What to do for ONE item, idempotent	`/tsys:onboard {type} --unattended`
Pre-flight gate	Must pass before tick 1 (creds, inputs)	`dark-factory-preflight.sh`
Stall policy	One item's failure stalls it, never halts the batch	gate fail / auth expiry → stall + continue

The clock is `/loop` (re-fires the tick after each returns; ends when the queue drains) or `ScheduleWakeup` (schedules the next tick + a delay). Validated 2026-06-11: a tick fired with **no human re-invocation**, processed the next item, and self-terminated when the queue emptied.

THE FIVE NON-NEGOTIABLE INVARIANTS

- **Idempotent ticks** — a half-done item is `in-progress`; the next tick continues it, never double-processes.
- **Stall, never halt** — the batch is as fragile as its pre-flight gate, not as its worst item.
- **Verify before irreversible** — trust-but-verify each item before any deploy (this caught a worktree bug).
- **Pre-flight gates loud** — fail before tick 1 if creds/inputs aren't ready; never waste a batch that can't finish.
- **The durable record is the goal-state file**, not the chat — terse tick output.

The operator cadence is the **morning review**: the loop runs the queue overnight, stalling any item it can't finish; the human reviews the results and clears the deploy gate in the morning. Zero 3 AM wake-ups — a stall is non-urgent until morning.

```
PRE-FLIGHT (once): gate must pass, or STOP before tick 1
EACH TICK: pick first pending → mark in-progress → run action → VERIFY
           → record done | stalled(+reason) → SELF-FIRE next tick
queue drains → write summary → STOP (no re-fire) ── operator does the morning review
```

The lesson flywheel — the factory improving the factory

Each onboarding **harvests lessons** into `memory.md` (60 numbered `L-NN` + 9 global `G-NN`); `/tsys:build` reads them before any stage runs, and `/tsys:ship` appends new ones after. The next onboarding starts smarter than the last — the factory gets faster and safer with every type.

PROMOTION: WHEN A LESSON BECOMES A SKILL

A lesson earns a callable skill when **all three** hold: **recurrence** (cited across ≥ 3 types), **high cost if wrong** (a financial / data-corruption / cloud-failure class error), and **mechanizable** (a deterministic decision tree or helper, not a per-case judgment). The loop owns creating the skill — it doesn't defer to a human — then retires the lesson into a pointer ("now enforced by the `<skill>` skill") and retrofits one existing consumer as proof it produces identical output with less code.

Skill	Promoted from	Prevents
`tsys-money-field`	money tree, cited across 10 types	the 100× scaling bug
`tsys-positional-hierarchical`	tc40 / tc33 / draftc / cbrt	slice + UUID + empty-table errors
`tsys-cobol-overpunch`	overpunch sign decode (L-51)	flipped-sign money corruption
`tsys-reconciliation`	the 5 record-count bases (L-54)	silently dropped records
`tsys-golden-match`	the Work-2 diff-triage batch	uncited parser edits

PROCESS GUARDS — THE FRESHEST LESSON BECOMES A GATE

Beyond skills, the sharpest lessons become **deterministic guards** — a script that fails the build if the mistake recurs. Three shipped in PR #211, each with a verified negative control: the **S3-prefix guard** (L-56 — a hyphenated type key must keep its hyphen across IAM ARN + OUTPUT_PREFIX + Bronze glob, or the Lambda 403s on its own trigger object or silently writes 0-row Bronze), the **pricing-drift guard**, and the **all-zero-Bronze guard**. The doctrine: turn the freshest lesson into a gate.

```
ONBOARD type → harvest L-NN lessons → memory.md ──→ next run reads FIRST
  └ lesson recurs ≥3× + high-cost + mechanizable → promote to SKILL or GUARD
                (money · overpunch · positional · reconciliation · golden-match)
```

Golden-parity sign-off — proving the translation

A structural check can pass while values diverge from legacy — the core risk of a T-SQL → Python translation. `/tsys:match` closes that gap: a 4-stage diff of the new output against the actual MSSQL/SSIS table dumps, the same fixtures locally and in the cloud.

```
specs/{type}/tests/input  —(1 parse)→  data/output/{type}  —(2 local compare)→  golden/
  raw TSYS file              parser+writer   Parquet          MatchService        GROUND TRUTH
  S3 landing  —(3 cloud)→  S3 stage  —(load)→  Databricks Bronze  —(4 bronze)→  GROUND TRUTH
```

The scoreboard lives at `match-state.yaml`: **26 of 28** matched types `signed`, ~160k rows value-for-value, **9 cited parser fixes, 0 uncited edits**. TDDF was the reference (8/8, catching 4 real parser bugs + a legacy cross-join). The batch's guiding principle: **the golden is evidence, not truth — the raw bytes plus the proc are truth.**

THE DIFF-TRIAGE TREE (THE TSYS-GOLDEN-MATCH SKILL)

Every dirty column is classified before anything is edited — the reflex "edit the parser until the diff vanishes" is the single most dangerous move in the program:

- **Representation** → a config edit only (naming, datetime, decimal, padding).
- **Fixture defect** → the golden side is wrong; exclude with a cited `known_divergence` — e.g. the varchar-coercion fingerprint (`golden == ours/100` on a DDL-varchar = SSMS coerced a routing number to a float), which recurred across 8 types.
- **Legacy manufacture** → T-SQL blank coercion (`convert(int,'')=0`); the proc citation decides whether the parser adopts it or Bronze stays faithful.
- **Parser bug** → the only branch that edits `src/core/`, and only with a proc / DDL / raw-byte citation. No citation → stall with proof.

The **2 stalled** types (gl-interface, tc33) are exactly this discipline in action: both stalled on fixture-provenance defects — a golden exported from a different day (gl-interface) and a 2025 golden against a 2026 input (tc33) — proven via the golden's own header row and kicked back to Kurv for re-export, rather than faked into a green.

The framework spine

- **Two ledgers**: `status.yaml` answers "is the type built?"; `match-state.yaml` answers "does its output match legacy 100%?"
- **Worktree isolation** (`tsys-worktree.sh`): each onboarding gets its own branch + git worktree; `/tsys:build` refuses to run if CWD doesn't match.
- **Archive lifecycle**: `/tsys:ship` writes Shape-A artifacts (DEFINE + DESIGN + BUILD_REPORT + SHIPPED, + AUTOMATION for onboard runs); Shape D adds a `CHANGE_{date}.md` per spec revision, preserving the original.
- **Observability** (L-28): FlowCheck `observe` reads the session transcript and emits measured token/model counts + an estimated cost — the ~8 h/file figure is measured, not asserted.